Main.go

```
package main

import (
"context"
"fmt"
"log"
"net/http"
"os"
"os/signal"
"syscall"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/database"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/kafka"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/redis"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/config"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/handlers"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/repositories"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/routes"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/services"
authclient "github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/auth-client"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/middleware"
"github.com/gin-gonic/gin"
"github.com/joho/godotenv"
"gorm.io/gorm"
)

// @title AA Core System - Valuation Service API
// @version 1.0
// @description Vehicle valuation service for AA Core System
// @contact.name API Support
// @contact.email support@aa.co.ke
// @host localhost:8084
```

```
// @BasePath /api/v1
// @securityDefinitions.apikey BearerAuth
// @in header
// @name Authorization
func main() {
// Load environment variables
if err := godotenv.Load(); err != nil {
log.Println("No .env file found, using system environment variables")
}

// Load configuration
cfg, err := config.LoadConfig("./configs")
if err != nil {
log.Fatalf("Failed to load config: %v", err)
}

// Set Gin mode
gin.SetMode(cfg.Server.Mode)

// Initialize database (only valuation database needed)
dbConfig := database.Config{
Host: cfg.Database.Host,
Port: cfg.Database.Port,
User: cfg.Database.User,
Password: cfg.Database.Password,
DBName: "aa_valuation_service", // Separate database for valuation service
SSLMode: cfg.Database.SSLMode,
MaxOpenConns: cfg.Database.MaxOpenConns,
MaxIdleConns: cfg.Database.MaxIdleConns,
ConnMaxLifetime: cfg.Database.ConnMaxLifetime,
}

db, err := database.NewPostgresConnection(dbConfig)
if err != nil {
log.Fatalf("Failed to connect to database: %v", err)
}

// Run migrations
if err := runMigrations(db); err != nil {
log.Fatalf("Failed to run migrations: %v", err)
}

// Initialize Redis for caching
redisConfig := redis.Config{
```

```
Host: cfg.Redis.Host,
Port: cfg.Redis.Port,
Password: cfg.Redis.Password,
DB: 3, // Different Redis DB for valuation service
PoolSize: cfg.Redis.PoolSize,
}

redisClient, err := redis.NewRedisClient(redisConfig)
if err != nil {
log.Fatalf("Failed to connect to Redis: %v", err)
}

// Initialize Kafka
kafkaConfig := kafka.Config{
Brokers: cfg.Kafka.Brokers,
Topics: map[string]string{
"user_events": "user-events",
"valuation_events": "valuation-events",
"partner_events": "partner-events",
"agent_events": "agent-events",
"valuation_type_events": "valuation-type-events",
"branch_events": "branch-events",
"vehicle_insurance_events": "vehicle-insurance-events",
"valuer_events": "valuer-events",
"valuation_assignment_events": "valuation-assignment-events",
"notification_events": "notification-events",
"valuation_request_events": "valuation-request-events",
"payment_events": "payment-events",
"external_payment_events": "external-payment-events",
},
}

kafkaProducer := kafka.NewProducer(kafkaConfig)
defer kafkaProducer.Close()

// Initialize Auth Service Client
authServiceURL := os.Getenv("AUTH_SERVICE_URL")
if authServiceURL == "" {
authServiceURL = "http://localhost:8083" // Your auth service port
}

log.Printf("🔐 Configuring auth client with URL: %s", authServiceURL)

authClientConfig := authclient.Config{
```

```
AuthServiceURL: authServiceURL,
ServiceID: "valuation-service",
ServiceVersion: "1.0",
SecretKey: cfg.JWT.Secret, // Same secret as auth service
Timeout: 5 * time.Second,
RedisClient: redisClient,
FallbackMode: true, // Enable fallback for resilience
}

authClient := authclient.NewAuthClient(authClientConfig)

// Test auth service connection
log.Printf("🔍 Testing auth service connection...")
if authClient.IsHealthy() {
log.Printf("✅ Auth service is healthy")
} else {
log.Printf("❌ Auth service is not responding")
}

// Initialize auth middleware
authMiddleware := middleware.NewAuthServiceMiddleware(authClient, true)

// Initialize repositories
customerRepo := repositories.NewCustomerRepository(db)
partnerRepo := repositories.NewPartnerRepository(db)
agentRepo := repositories.NewAgentRepository(db)
valuationTypeRepo := repositories.NewValuationTypeRepository(db)
branchRepo := repositories.NewBranchRepository(db)
vehicleInsuranceRepo := repositories.NewVehicleInsuranceRepository(db)
valuerRepo := repositories.NewValuerRepository(db)
valuationRequestRepo := repositories.NewValuationRequestRepository(db)
valuationInvoiceRepo := repositories.NewValuationInvoiceRepository(db)
valuationPaymentRepo := repositories.NewValuationPaymentRepository(db)
// Add other repositories as you implement them

// Initialize services
customerService := services.NewCustomerService(customerRepo, kafkaProducer)
partnerService := services.NewPartnerService(partnerRepo, kafkaProducer)
agentService := services.NewAgentService(agentRepo, partnerRepo, kafkaProducer)
valuationTypeService := services.NewValuationTypeService(valuationTypeRepo,
kafkaProducer)
branchService := services.NewBranchService(branchRepo, kafkaProducer)
```

```
vehicleInsuranceService :=
services.NewVehicleInsuranceService(vehicleInsuranceRepo, customerRepo,
kafkaProducer)
valuerService := services.NewValuerService(valuerRepo, kafkaProducer)
valuationRequestService := services.NewValuationRequestService(
valuationRequestRepo,
customerRepo,
partnerRepo,
agentRepo,
valuerRepo,
kafkaProducer,
)
valuationInvoiceService :=
services.NewValuationInvoiceService(valuationInvoiceRepo, valuationRequestRepo,
kafkaProducer)
valuationPaymentService :=
services.NewValuationPaymentService(valuationPaymentRepo, valuationInvoiceRepo,
kafkaProducer)
// Add other services as you implement them

// Initialize handlers
customerHandler := handlers.NewCustomerHandler(customerService)
partnerHandler := handlers.NewPartnerHandler(partnerService)
agentHandler := handlers.NewAgentHandler(agentService)
valuationTypeHandler := handlers.NewValuationTypeHandler(valuationTypeService)
branchHandler := handlers.NewBranchHandler(branchService)
vehicleInsuranceHandler :=
handlers.NewVehicleInsuranceHandler(vehicleInsuranceService)
valuerHandler := handlers.NewValuerHandler(valuerService)
valuationRequestHandler :=
handlers.NewValuationRequestHandler(valuationRequestService)
valuationInvoiceHandler :=
handlers.NewValuationInvoiceHandler(valuationInvoiceService)
valuationPaymentHandler :=
handlers.NewValuationPaymentHandler(valuationPaymentService)
// Add other handlers as you implement them

// Initialize Gin router
router := gin.New()
router.Use(gin.Recovery())

// Setup routes with auth service middleware
routes.SetupRoutes(
router,
```

```
customerHandler,
partnerHandler,
agentHandler,
valuationTypeHandler,
branchHandler,
vehicleInsuranceHandler,
valuerHandler,
valuationRequestHandler,
valuationInvoiceHandler,
valuationPaymentHandler,
authMiddleware,
)

// Start Kafka consumers in background
go startKafkaConsumers(
kafkaConfig,
customerService,
partnerService,
agentService,
valuationTypeService,
branchService,
vehicleInsuranceService,
valuerService,
valuationRequestService,
valuationInvoiceService,
valuationPaymentService,
)

// Start auth service health monitoring
go monitorAuthService(authClient)

// Start scheduled tasks
go startScheduledTasks(vehicleInsuranceService)

// Initialize default data
go func() {
time.Sleep(5 * time.Second) // Wait for service to be ready
if err := initializeDefaultData(valuationTypeService, branchService); err != nil {
log.Printf("Warning: Failed to initialize default data: %v", err)
}
}()

port := os.Getenv("PORT")
if port == "" {
```

```
port = "8084" // Different port for valuation service
}

// Create HTTP server
srv := &http.Server{
Addr: ":" + port,
Handler: router,
ReadTimeout: cfg.Server.ReadTimeout,
WriteTimeout: cfg.Server.WriteTimeout,
}

// Start server in a goroutine
go func() {
log.Printf("Starting valuation service on port %s", port)
log.Printf("Using auth service at: %s", authServiceURL)
if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
log.Fatalf("Failed to start server: %v", err)
}
}()

// Wait for interrupt signal to gracefully shutdown the server
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

log.Println("Shutting down valuation service...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
log.Fatalf("Server forced to shutdown: %v", err)
}

log.Println("Valuation service shutdown complete")
}

// runMigrations handles database migrations for valuation service
func runMigrations(db *gorm.DB) error {
log.Println("Running valuation service migrations...")

err := db.AutoMigrate(
&models.Customer{},
&models.Partners{},
```

```
&models.Agents{},
&models.ValuationTypes{},
&models.Branches{},
&models.VehicleInsurances{},
&models.Valuer{},
&models.ValuationRequests{},
&models.ValuationInvoices{},
&models.ValuationPayments{},
)
if err != nil {
return err
}

log.Println("Valuation service migrations completed successfully")
return nil
}

// initializeDefaultData initializes default valuation types and branches
func initializeDefaultData(valuationTypeService *services.ValuationTypeService,
branchService *services.BranchService) error {
log.Println("Initializing default valuation data...")

// Initialize default valuation types
if err := valuationTypeService.InitializeDefaultValuationTypes(); err != nil {
return fmt.Errorf("failed to initialize default valuation types: %v", err)
}

// Initialize default branches
if err := branchService.InitializeDefaultBranches(); err != nil {
return fmt.Errorf("failed to initialize default branches: %v", err)
}

log.Println("Default valuation data initialized successfully")
return nil
}

// startScheduledTasks starts background scheduled tasks
func startScheduledTasks(vehicleInsuranceService *services.VehicleInsuranceService)
{
// Check for expiry notifications every 24 hours
ticker := time.NewTicker(24 * time.Hour)
defer ticker.Stop()

// Run immediately on startup
```

```
if err := vehicleInsuranceService.CheckExpiryNotifications(); err != nil {
log.Printf("Failed to check expiry notifications: %v", err)
}

for {
select {
case <-ticker.C:
if err := vehicleInsuranceService.CheckExpiryNotifications(); err != nil {
log.Printf("Failed to check expiry notifications: %v", err)
}
}
}
}

// startKafkaConsumers handles Kafka event consumption
func startKafkaConsumers(
kafkaConfig kafka.Config,
customerService *services.CustomerService,
partnerService *services.PartnerService,
agentService *services.AgentService,
valuationTypeService *services.ValuationTypeService,
branchService *services.BranchService,
vehicleInsuranceService *services.VehicleInsuranceService,
valuerService *services.ValuerService,
valuationRequestService *services.ValuationRequestService,
valuationInvoiceService *services.ValuationInvoiceService,
valuationPaymentService *services.ValuationPaymentService,
) {
consumer := kafka.NewConsumer(kafkaConfig, "valuation-service")

// Handle user events from user-management service
go func() {
err := consumer.ConsumeEvents("user_events", func(event kafka.Event) error {
switch event.Type {
case "user.registered":
return customerService.HandleCustomerCreated(event)
case "user.email_verified":
return customerService.HandleCustomerVerified(event)
default:
log.Printf("Unhandled user event type: %s", event.Type)
}
return nil
})
if err != nil {
```

```
log.Printf("Failed to consume user events: %v", err)
}
}()

// Handle partner events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("partner_events", func(event kafka.Event) error {
switch event.Type {
case "partner.created":
log.Printf("Partner created: %v", event.Data)
case "partner.updated":
log.Printf("Partner updated: %v", event.Data)
case "partner.deleted":
log.Printf("Partner deleted: %v", event.Data)
default:
log.Printf("Unhandled partner event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume partner events: %v", err)
}
}()

// Handle agent events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("agent_events", func(event kafka.Event) error {
switch event.Type {
case "agent.created":
log.Printf("Agent created: %v", event.Data)
case "agent.updated":
log.Printf("Agent updated: %v", event.Data)
case "agent.deleted":
log.Printf("Agent deleted: %v", event.Data)
default:
log.Printf("Unhandled agent event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume agent events: %v", err)
}
}()
```

```
// Handle valuation type events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("valuation_type_events", func(event kafka.Event)
error {
switch event.Type {
case "valuation_type.created":
log.Printf("Valuation type created: %v", event.Data)
case "valuation_type.updated":
log.Printf("Valuation type updated: %v", event.Data)
case "valuation_type.deleted":
log.Printf("Valuation type deleted: %v", event.Data)
default:
log.Printf("Unhandled valuation type event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume valuation type events: %v", err)
}
}()

// Handle branch events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("branch_events", func(event kafka.Event) error {
switch event.Type {
case "branch.created":
log.Printf("Branch created: %v", event.Data)
case "branch.updated":
log.Printf("Branch updated: %v", event.Data)
case "branch.deleted":
log.Printf("Branch deleted: %v", event.Data)
default:
log.Printf("Unhandled branch event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume branch events: %v", err)
}
}()

// Handle vehicle insurance events (if needed for cross-service communication)
go func() {
```

```
err := consumer.ConsumeEvents("vehicle_insurance_events", func(event kafka.Event)
error {
switch event.Type {
case "vehicle_insurance.created":
log.Printf("Vehicle insurance created: %v", event.Data)
case "vehicle_insurance.updated":
log.Printf("Vehicle insurance updated: %v", event.Data)
case "vehicle_insurance.deleted":
log.Printf("Vehicle insurance deleted: %v", event.Data)
default:
log.Printf("Unhandled vehicle insurance event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume vehicle insurance events: %v", err)
}
}()

// Handle valuer events (if needed for cross-service communication)
go func() {
err := consumer.ConsumeEvents("valuer_events", func(event kafka.Event) error {
switch event.Type {
case "valuer.created":
log.Printf("Valuer created: %v", event.Data)
case "valuer.updated":
log.Printf("Valuer updated: %v", event.Data)
case "valuer.deleted":
log.Printf("Valuer deleted: %v", event.Data)
default:
log.Printf("Unhandled valuer event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume valuer events: %v", err)
}
}()

// Handle valuation assignment events
go func() {
err := consumer.ConsumeEvents("valuation_assignment_events", func(event
kafka.Event) error {
switch event.Type {
```

```
case "valuer.assigned_to_request":
log.Printf("Valuer assigned to request: %v", event.Data)
default:
log.Printf("Unhandled valuation assignment event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume valuation assignment events: %v", err)
}
}()

// Handle valuation request events - you can add more event handlers here
go func() {
err := consumer.ConsumeEvents("valuation_request_events", func(event kafka.Event)
error {
switch event.Type {
case "vehicle.created":
// Handle vehicle creation if needed
log.Printf("Vehicle created event received: %v", event.Data)
case "vehicle.updated":
// Handle vehicle updates if needed
log.Printf("Vehicle updated event received: %v", event.Data)
default:
log.Printf("Unhandled valuation event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume valuation events: %v", err)
}
}()

// Handle payment events - for updating invoice status when payments are received
go func() {
err := consumer.ConsumeEvents("payment_events", func(event kafka.Event) error {
switch event.Type {
case "payment.completed":
// Update invoice status when payment is completed
if invoiceNumber, ok := event.Data["invoice_number"].(string); ok {
log.Printf("Payment completed for invoice: %s", invoiceNumber)
// You could implement auto-update of invoice status here
// valuationInvoiceService.UpdateInvoiceStatusByNumber(invoiceNumber, "paid")
}
```

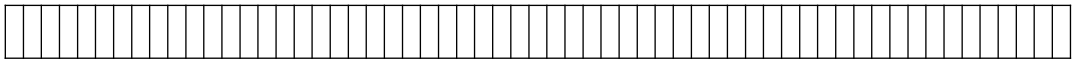
```
default:
log.Printf("Unhandled payment event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume payment events: %v", err)
}
}()

// Handle external system events - for payment gateway integrations
go func() {
err := consumer.ConsumeEvents("external_payment_events", func(event kafka.Event)
error {
switch event.Type {
case "payment_gateway.payment_completed":
// Process payment from external gateway
if invoiceNumber, ok := event.Data["invoice_number"].(string); ok {
if amount, ok := event.Data["amount"].(float64); ok {
if transactionID, ok := event.Data["transaction_id"].(string); ok {
_, err :=
valuationPaymentService.ProcessPaymentFromExternalSystem(invoiceNumber,
transactionID, amount)
if err != nil {
log.Printf("Failed to process external payment: %v", err)
return err
}
log.Printf("Successfully processed external payment for invoice: %s", invoiceNumber)
}
}
}
case "payment_gateway.payment_failed":
// Handle failed payment from external gateway
if invoiceNumber, ok := event.Data["invoice_number"].(string); ok {
log.Printf("External payment failed for invoice: %s", invoiceNumber)
// You might want to notify the customer or update some status
}
case "payment_gateway.refund_processed":
// Handle refund from external gateway
if receiptNumber, ok := event.Data["receipt_number"].(string); ok {
if reason, ok := event.Data["reason"].(string); ok {
payment, err :=
valuationPaymentService.GetValuationPaymentByReceiptNumber(receiptNumber)
if err != nil {
```

```
log.Printf("Payment not found for receipt: %s", receiptNumber)
return err
}
_, err = valuationPaymentService.RefundPayment(payment.ID, reason)
if err != nil {
log.Printf("Failed to process refund for receipt: %s", receiptNumber)
return err
}
log.Printf("Successfully processed refund for receipt: %s", receiptNumber)
}
}
default:
log.Printf("Unhandled external payment event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume external payment events: %v", err)
}
}()
}

// monitorAuthService monitors the health of the auth service
func monitorAuthService(authClient *authclient.AuthClient) {
ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()

for range ticker.C {
if !authClient.IsHealthy() {
log.Printf("Warning: Auth service is not responding")
}
}
}
```



Routes.go

```
package routes

import (
"time"
```

```
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/handlers"
authMiddleware "github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/middleware"
"github.com/gin-gonic/gin"
swaggerFiles "github.com/swaggo/files"
ginSwagger "github.com/swaggo/gin-swagger"
)

func SetupRoutes(
r *gin.Engine,
customerHandler *handlers.CustomerHandler,
partnerHandler *handlers.PartnerHandler,
agentHandler *handlers.AgentHandler,
valuationTypeHandler *handlers.ValuationTypeHandler,
branchHandler *handlers.BranchHandler,
vehicleInsuranceHandler *handlers.VehicleInsuranceHandler,
valuerHandler *handlers.ValuerHandler,
valuationRequestHandler *handlers.ValuationRequestHandler,
valuationInvoiceHandler *handlers.ValuationInvoiceHandler,
valuationPaymentHandler *handlers.ValuationPaymentHandler,
authServiceMiddleware *authMiddleware.AuthServiceMiddleware, // Updated parameter type
) {
// Add CORS middleware
r.Use(middleware.CORSMiddleware())

// Add logging middleware
r.Use(middleware.LoggingMiddleware())

// Health check (no authentication required)
r.GET("/health", func(c *gin.Context) {
c.JSON(200, gin.H{
"status": "healthy",
"service": "valuation-service",
"timestamp": time.Now(),
})
})

// Swagger documentation
r.GET("/docs/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))
```

```
// API routes
api := r.Group("/api/v1")

// Public routes (reference data that doesn't require authentication)
public := api.Group("/public")
{
// Public valuation types for dropdown/reference
public.GET("/valuation-types", valuationTypeHandler.GetActiveValuationTypes) //
GET /api/v1/public/valuation-types

// Public branches for dropdown/reference
public.GET("/branches", branchHandler.GetActiveBranches) // GET
/api/v1/public/branches

// Public valuers for dropdown/reference
public.GET("/valuers", valuerHandler.GetActiveValuers) // GET /api/v1/public/valuers
}

// Protected routes (using auth service)
protected := api.Group("/")
protected.Use(authServiceMiddleware.RequireAuth()) // Updated to use auth service
{
// Customer operations
customers := protected.Group("/customers")
{
// Basic CRUD operations
customers.GET("", customerHandler.GetAllCustomers) // GET /api/v1/customers?
limit=10&offset=0
customers.POST("", customerHandler.CreateCustomer) // POST /api/v1/customers
customers.GET("/:id", customerHandler.GetCustomerByID) // GET
/api/v1/customers/{id}
customers.PUT("/:id", customerHandler.UpdateCustomer) // PUT
/api/v1/customers/{id}
customers.DELETE("/:id", customerHandler.DeleteCustomer) // DELETE
/api/v1/customers/{id}
customers.GET("/search", customerHandler.SearchCustomers) // GET
/api/v1/customers/search?q=john&limit=10&offset=0
customers.GET("/status/:status", customerHandler.GetCustomersByStatus) // GET
/api/v1/customers/status/active?limit=10&offset=0
}

// Partner operations
partners := protected.Group("/partners")
{
```

```
// Basic CRUD operations
partners.GET("", partnerHandler.GetAllPartners) // GET /api/v1/partners?
limit=10&offset=0
partners.POST("", partnerHandler.CreatePartner) // POST /api/v1/partners
partners.GET("/:id", partnerHandler.GetPartnerByID) // GET /api/v1/partners/{id}
partners.PUT("/:id", partnerHandler.UpdatePartner) // PUT /api/v1/partners/{id}
partners.DELETE("/:id", partnerHandler.DeletePartner) // DELETE /api/v1/partners/{id}
partners.GET("/search", partnerHandler.SearchPartners) // GET
/api/v1/partners/search?q=company&limit=10&offset=0
partners.GET("/type/:type", partnerHandler.GetPartnersByType) // GET
/api/v1/partners/type/insurance?limit=10&offset=0
partners.GET("/status/:status", partnerHandler.GetPartnersByStatus) // GET
/api/v1/partners/status/active?limit=10&offset=0
partners.GET("/:id/agents", partnerHandler.GetPartnerWithAgents) // GET
/api/v1/partners/{id}/agents
}

// Agent operations
agents := protected.Group("/agents")
{
// Basic CRUD operations
agents.GET("", agentHandler.GetAllAgents) // GET /api/v1/agents?limit=10&offset=0
agents.POST("", agentHandler.CreateAgent) // POST /api/v1/agents
agents.GET("/:id", agentHandler.GetAgentByID) // GET /api/v1/agents/{id}
agents.PUT("/:id", agentHandler.UpdateAgent) // PUT /api/v1/agents/{id}
agents.DELETE("/:id", agentHandler.DeleteAgent) // DELETE /api/v1/agents/{id}
agents.GET("/search", agentHandler.SearchAgents) // GET /api/v1/agents/search?
q=agent&limit=10&offset=0
agents.GET("/type/:type", agentHandler.GetAgentsByType) // GET
/api/v1/agents/type/individual?limit=10&offset=0
agents.GET("/status/:status", agentHandler.GetAgentsByStatus) // GET
/api/v1/agents/status/active?limit=10&offset=0
agents.GET("/partner/:partner_id", agentHandler.GetAgentsByPartner) // GET
/api/v1/agents/partner/{partner_id}?limit=10&offset=0
agents.GET("/:id/valuation-requests", agentHandler.GetAgentWithValuationRequests)
// GET /api/v1/agents/{id}/valuation-requests
}

// Valuation Types operations
valuationTypes := protected.Group("/valuation-types")
{
// Basic CRUD operations
valuationTypes.GET("", valuationTypeHandler.GetAllValuationTypes) // GET
/api/v1/valuation-types?limit=10&offset=0
```

```
valuationTypes.POST("", valuationTypeHandler.CreateValuationType) // POST
/api/v1/valuation-types
valuationTypes.GET("/:id", valuationTypeHandler.GetValuationTypeByID) // GET
/api/v1/valuation-types/{id}
valuationTypes.PUT("/:id", valuationTypeHandler.UpdateValuationType) // PUT
/api/v1/valuation-types/{id}
valuationTypes.DELETE("/:id", valuationTypeHandler.DeleteValuationType) //
DELETE /api/v1/valuation-types/{id}
valuationTypes.GET("/search", valuationTypeHandler.SearchValuationTypes) // GET
/api/v1/valuation-types/search?q=insurance&limit=10&offset=0
valuationTypes.GET("/status/:status",
valuationTypeHandler.GetValuationTypesByStatus) // GET
/api/v1/valuation-types/status/active?limit=10&offset=0
valuationTypes.GET("/active", valuationTypeHandler.GetActiveValuationTypes) //
GET /api/v1/valuation-types/active?limit=10&offset=0
valuationTypes.GET("/name/:name",
valuationTypeHandler.GetValuationTypeByName) // GET
/api/v1/valuation-types/name/{name}
}

// Branch operations
branches := protected.Group("/branches")
{
// Basic CRUD operations
branches.GET("", branchHandler.GetAllBranches) // GET /api/v1/branches?
limit=10&offset=0
branches.POST("", branchHandler.CreateBranch) // POST /api/v1/branches
branches.GET("/:id", branchHandler.GetBranchByID) // GET /api/v1/branches/{id}
branches.PUT("/:id", branchHandler.UpdateBranch) // PUT /api/v1/branches/{id}
branches.DELETE("/:id", branchHandler.DeleteBranch) // DELETE
/api/v1/branches/{id}
branches.GET("/search", branchHandler.SearchBranches) // GET
/api/v1/branches/search?q=nairobi&limit=10&offset=0
branches.GET("/status/:status", branchHandler.GetBranchesByStatus) // GET
/api/v1/branches/status/active?limit=10&offset=0
branches.GET("/active", branchHandler.GetActiveBranches) // GET
/api/v1/branches/active?limit=10&offset=0
branches.GET("/location", branchHandler.GetBranchesByLocation) // GET
/api/v1/branches/location?location=nairobi&limit=10&offset=0
branches.GET("/name/:name", branchHandler.GetBranchByName) // GET
/api/v1/branches/name/{name}
branches.GET("/stats", branchHandler.GetBranchStats) // GET /api/v1/branches/stats
}
```

```
// Vehicle Insurance operations
vehicleInsurances := protected.Group("/vehicle-insurances")
{
// Basic CRUD operations
vehicleInsurances.GET("", vehicleInsuranceHandler.GetAllVehicleInsurances) // GET
/api/v1/vehicle-insurances?limit=10&offset=0
vehicleInsurances.POST("", vehicleInsuranceHandler.CreateVehicleInsurance) //
POST /api/v1/vehicle-insurances
vehicleInsurances.GET("/:id", vehicleInsuranceHandler.GetVehicleInsuranceByID) //
GET /api/v1/vehicle-insurances/{id}
vehicleInsurances.PUT("/:id", vehicleInsuranceHandler.UpdateVehicleInsurance) //
PUT /api/v1/vehicle-insurances/{id}
vehicleInsurances.DELETE("/:id", vehicleInsuranceHandler.DeleteVehicleInsurance) //
DELETE /api/v1/vehicle-insurances/{id}
vehicleInsurances.GET("/search", vehicleInsuranceHandler.SearchVehicleInsurances)
// GET /api/v1/vehicle-insurances/search?q=policy&limit=10&offset=0
vehicleInsurances.GET("/status/:status",
vehicleInsuranceHandler.GetVehicleInsurancesByStatus) // GET /api/v1/vehicle-
insurances/status/active?limit=10&offset=0
vehicleInsurances.GET("/customer/:customer_id",
vehicleInsuranceHandler.GetVehicleInsurancesByCustomer) // GET /api/v1/vehicle-
insurances/customer/{customer_id}?limit=10&offset=0
vehicleInsurances.GET("/company",
vehicleInsuranceHandler.GetVehicleInsurancesByCompany) // GET /api/v1/vehicle-
insurances/company?company=jubilee&limit=10&offset=0
vehicleInsurances.GET("/expiring",
vehicleInsuranceHandler.GetExpiringVehicleInsurances) // GET /api/v1/vehicle-
insurances/expiring?days=30&limit=10&offset=0
vehicleInsurances.GET("/policy/:policy_number",
vehicleInsuranceHandler.GetVehicleInsuranceByPolicyNumber) // GET
/api/v1/vehicle-insurances/policy/{policy_number}
vehicleInsurances.GET("/stats", vehicleInsuranceHandler.GetVehicleInsuranceStats) //
GET /api/v1/vehicle-insurances/stats
}

// Valuer operations
valuers := protected.Group("/valuers")
{
// Basic CRUD operations
valuers.GET("", valuerHandler.GetAllValuers) // GET /api/v1/valuers?limit=10&offset=0
valuers.POST("", valuerHandler.CreateValuer) // POST /api/v1/valuers
valuers.GET("/:id", valuerHandler.GetValuerByID) // GET /api/v1/valuers/{id}
valuers.PUT("/:id", valuerHandler.UpdateValuer) // PUT /api/v1/valuers/{id}
valuers.DELETE("/:id", valuerHandler.DeleteValuer) // DELETE /api/v1/valuers/{id}
```

```
valuers.GET("/search", valuerHandler.SearchValuers) // GET /api/v1/valuers/search?q=john&limit=10&offset=0
valuers.GET("/status/:status", valuerHandler.GetValuersByStatus) // GET /api/v1/valuers/status/active?limit=10&offset=0
valuers.GET("/active", valuerHandler.GetActiveValuers) // GET /api/v1/valuers/active?limit=10&offset=0
valuers.GET("/location", valuerHandler.GetValuersByLocation) // GET /api/v1/valuers/location?location=nairobi&limit=10&offset=0
valuers.GET("/phone/:phone", valuerHandler.GetValuerByPhone) // GET /api/v1/valuers/phone/{phone}
valuers.GET("/:id/valuation-requests", valuerHandler.GetValuerWithValuationRequests) // GET /api/v1/valuers/{id}/valuation-requests
valuers.GET("/stats", valuerHandler.GetValuerStats) // GET /api/v1/valuers/stats
valuers.GET("/:id/workload", valuerHandler.GetValuerWorkload) // GET /api/v1/valuers/{id}/workload
}

// Valuation Requests
valuationRequests := protected.Group("/valuation-requests")
{
// Basic CRUD operations
valuationRequests.GET("/", valuationRequestHandler.GetAllValuationRequests) // GET /api/v1/valuation-requests?limit=10&offset=0
valuationRequests.POST("/", valuationRequestHandler.CreateValuationRequest) // POST /api/v1/valuation-requests
valuationRequests.GET("/:id", valuationRequestHandler.GetValuationRequestByID) // GET /api/v1/valuation-requests/{id}
valuationRequests.PUT("/:id", valuationRequestHandler.UpdateValuationRequest) // PUT /api/v1/valuation-requests/{id}
valuationRequests.DELETE("/:id", valuationRequestHandler.DeleteValuationRequest) // DELETE /api/v1/valuation-requests/{id}

// Search and filter operations
valuationRequests.GET("/search", valuationRequestHandler.SearchValuationRequests) // GET /api/v1/valuation-requests/search?q=john&limit=10&offset=0
valuationRequests.GET("/status/:status", valuationRequestHandler.GetValuationRequestsByStatus) // GET /api/v1/valuation-requests/status/pending?limit=10&offset=0

// Relationship-based queries
```

```
valuationRequests.GET("/customer/:customer_id",
valuationRequestHandler.GetValuationRequestsByCustomer) // GET
/api/v1/valuation-requests/customer/{customer_id}?limit=10&offset=0
valuationRequests.GET("/partner/:partner_id",
valuationRequestHandler.GetValuationRequestsByPartner) // GET /api/v1/valuation-
requests/partner/{partner_id}?limit=10&offset=0
valuationRequests.GET("/agent/:agent_id",
valuationRequestHandler.GetValuationRequestsByAgent) // GET /api/v1/valuation-
requests/agent/{agent_id}?limit=10&offset=0
valuationRequests.GET("/valuer/:valuer_id",
valuationRequestHandler.GetValuationRequestsByValuer) // GET /api/v1/valuation-
requests/valuer/{valuer_id}?limit=10&offset=0

// Special operations
valuationRequests.PUT("/:id/assign-valuer",
valuationRequestHandler.AssignValuerToRequest) // PUT
/api/v1/valuation-requests/{id}/assign-valuer
valuationRequests.PUT("/:id/status", valuationRequestHandler.UpdateRequestStatus)
// PUT /api/v1/valuation-requests/{id}/status
}

// Valuation Invoice operations
valuationInvoices := protected.Group("/valuation-invoices")
{
// Basic CRUD operations
valuationInvoices.GET("/", valuationInvoiceHandler.GetAllValuationInvoices) // GET
/api/v1/valuation-invoices?limit=10&offset=0
valuationInvoices.POST("/", valuationInvoiceHandler.CreateValuationInvoice) //
POST /api/v1/valuation-invoices
valuationInvoices.GET("/:id", valuationInvoiceHandler.GetValuationInvoiceByID) //
GET /api/v1/valuation-invoices/{id}
valuationInvoices.PUT("/:id", valuationInvoiceHandler.UpdateValuationInvoice) // PUT
/api/v1/valuation-invoices/{id}
valuationInvoices.DELETE("/:id", valuationInvoiceHandler.DeleteValuationInvoice) //
DELETE /api/v1/valuation-invoices/{id}

// Search and filter operations
valuationInvoices.GET("/search", valuationInvoiceHandler.SearchValuationInvoices) //
GET /api/v1/valuation-invoices/search?q=INV001&limit=10&offset=0
valuationInvoices.GET("/status/:status",
valuationInvoiceHandler.GetValuationInvoicesByStatus) // GET /api/v1/valuation-
invoices/status/paid?limit=10&offset=0
```

```
valuationInvoices.GET("/invoice-number/:invoice_number", valuationInvoiceHandler.GetValuationInvoiceByInvoiceNumber) // GET /api/v1/valuation-invoices/invoice-number/{invoice_number}

// Relationship-based queries
valuationInvoices.GET("/valuation-request/:valuation_request_id", valuationInvoiceHandler.GetValuationInvoicesByValuationRequest) // GET /api/v1/valuation-invoices/valuation-request/{valuation_request_id}?limit=10&offset=0

// Status-specific endpoints
valuationInvoices.GET("/paid", valuationInvoiceHandler.GetPaidInvoices) // GET /api/v1/valuation-invoices/paid?limit=10&offset=0
valuationInvoices.GET("/unpaid", valuationInvoiceHandler.GetUnpaidInvoices) // GET /api/v1/valuation-invoices/unpaid?limit=10&offset=0

// Date and amount range queries
valuationInvoices.GET("/date-range", valuationInvoiceHandler.GetInvoicesByDateRange) // GET /api/v1/valuation-invoices/date-range?start_date=2024-01-01&end_date=2024-12-31&limit=10&offset=0

// Special operations
valuationInvoices.PUT("/:id/status", valuationInvoiceHandler.UpdateInvoiceStatus) // PUT /api/v1/valuation-invoices/{id}/status

// Statistics and reporting
valuationInvoices.GET("/statistics", valuationInvoiceHandler.GetInvoiceStatistics) // GET /api/v1/valuation-invoices/statistics
}

// Valuation Payment operations
valuationPayments := protected.Group("/valuation-payments")
{
// Basic CRUD operations
valuationPayments.GET("/", valuationPaymentHandler.GetAllValuationPayments) // GET /api/v1/valuation-payments?limit=10&offset=0
valuationPayments.POST("/", valuationPaymentHandler.CreateValuationPayment) // POST /api/v1/valuation-payments
valuationPayments.GET("/:id", valuationPaymentHandler.GetValuationPaymentByID) // GET /api/v1/valuation-payments/{id}
valuationPayments.PUT("/:id", valuationPaymentHandler.UpdateValuationPayment) // PUT /api/v1/valuation-payments/{id}
```

```
valuationPayments.DELETE("/:id", valuationPaymentHandler.DeleteValuationPayment) // DELETE /api/v1/valuation-payments/{id}

// Search and filter operations
valuationPayments.GET("/search", valuationPaymentHandler.SearchValuationPayments) // GET /api/v1/valuation-payments/search?q=RCT001&limit=10&offset=0
valuationPayments.GET("/status/:status", valuationPaymentHandler.GetValuationPaymentsByStatus) // GET /api/v1/valuation-payments/status/paid?limit=10&offset=0
valuationPayments.GET("/receipt-number/:receipt_number", valuationPaymentHandler.GetValuationPaymentByReceiptNumber) // GET /api/v1/valuation-payments/receipt-number/{receipt_number}

// Relationship-based queries
valuationPayments.GET("/invoice/:invoice_number", valuationPaymentHandler.GetValuationPaymentsByInvoice) // GET /api/v1/valuation-payments/invoice/{invoice_number}?limit=10&offset=0

// Date and amount range queries
valuationPayments.GET("/date-range", valuationPaymentHandler.GetPaymentsByDateRange) // GET /api/v1/valuation-payments/date-range?start_date=2024-01-01&end_date=2024-12-31&limit=10&offset=0
valuationPayments.GET("/amount-range", valuationPaymentHandler.GetPaymentsByAmountRange) // GET /api/v1/valuation-payments/amount-range?min_amount=100&max_amount=1000&limit=10&offset=0

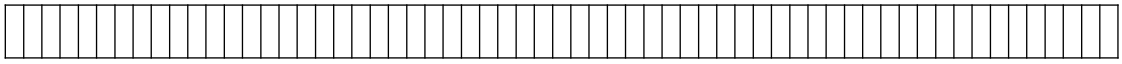
// Special operations
valuationPayments.PUT("/:id/status", valuationPaymentHandler.UpdatePaymentStatus) // PUT /api/v1/valuation-payments/{id}/status
valuationPayments.PUT("/:id/refund", valuationPaymentHandler.RefundPayment) // PUT /api/v1/valuation-payments/{id}/refund

// Statistics and reporting
valuationPayments.GET("/statistics", valuationPaymentHandler.GetPaymentStatistics) // GET /api/v1/valuation-payments/statistics
valuationPayments.GET("/recent", valuationPaymentHandler.GetRecentPayments) // GET /api/v1/valuation-payments/recent?limit=20
valuationPayments.GET("/total-amount", valuationPaymentHandler.GetTotalPaymentsAmount) // GET /api/v1/valuation-payments/total-amount
```

```
valuationPayments.GET("/monthly-total",
valuationPaymentHandler.GetMonthlyPaymentsTotal) // GET /api/v1/valuation-
payments/monthly-total?year=2024&month=12
}

// Admin only routes
admin := protected.Group("/admin")
admin.Use(authServiceMiddleware.RequireRole("admin", "super_admin")) // Updated
to use auth service
{
// Admin customer operations
// admin.GET("/customers/analytics", customerHandler.GetCustomerAnalytics)
// admin.POST("/customers/bulk-import", customerHandler.BulkImportCustomers)

}
}
}
```



Model

```
package models

import (
"time"

"github.com/google/uuid"
"gorm.io/gorm"
)

// Enum types for better type safety
type PartnerType string
type AgentType string
type StatusType string

const (
// Partner types
PartnerTypeInsurance PartnerType = "insurance"
PartnerTypeBank PartnerType = "bank"
PartnerTypeDealer PartnerType = "dealer"
PartnerTypeOther PartnerType = "other"

// Agent types
```

```
AgentTypeIndividual AgentType = "individual"
AgentTypeCorporate AgentType = "corporate"

// Status types
StatusActive StatusType = "active"
StatusInactive StatusType = "inactive"
StatusPending StatusType = "pending"
StatusApproved StatusType = "approved"
StatusRejected StatusType = "rejected"
StatusPaid StatusType = "paid"
StatusUnpaid StatusType = "unpaid"
)

// Customer represents customer data for valuation service
type Customer struct {
ID string `json:"id" gorm:"type:uuid;primary_key"`
Email string `json:"email" gorm:"not null;unique"`
Phone string `json:"phone" gorm:"not null;unique"`
FirstName string `json:"first_name" gorm:"not null"`
LastName string `json:"last_name" gorm:"not null"`
Status string `json:"status" gorm:"not null"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:CustomerID"`
VehicleInsurances []VehicleInsurances `json:"vehicle_insurances"
gorm:"foreignKey:CustomerID"`
}

// Partners represents partner organizations
type Partners struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
PartnerType PartnerType `json:"partner_type" gorm:"not null"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
CompanyPhone string `json:"company_phone" gorm:"not null"`
CompanyEmail string `json:"company_email" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
```

```
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Agents []Agents `json:"agents" gorm:"foreignKey:PartnerID"`
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:PartnerID"`
}

// Agents represents partner agents
type Agents struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
AgentType AgentType `json:"agent_type" gorm:"not null"`
PartnerID string `json:"partner_id" gorm:"not null"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
CompanyPhone string `json:"company_phone" gorm:"not null"`
CompanyEmail string `json:"company_email" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Partner Partners `json:"partner" gorm:"foreignKey:PartnerID"`
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:AgentID"`
}

// ValuationTypes represents different types of valuations
type ValuationTypes struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null;unique"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt time.Time `json:"deleted_at"`
}

// Branches represents company branches
type Branches struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
Location string `json:"location" gorm:"not null"`
```

```
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
}

// VehicleInsurances represents vehicle insurance information
type VehicleInsurances struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InsuranceCompany string `json:"insurance_company" gorm:"not null"`
PolicyNumber string `json:"policy_number" gorm:"unique;not null"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
CustomerID string `json:"customer_id" gorm:"not null"`
ExpiryDate time.Time `json:"expiry_date" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
}

// Valuer represents professional valuers
type Valuer struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
Phone string `json:"phone" gorm:"not null"`
Location string `json:"location" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:ValuerID"`
}

// ValuationRequests represents valuation requests
type ValuationRequests struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
PartnerID *string `json:"partner_id,omitempty"`
AgentID *string `json:"agent_id,omitempty"`
CustomerID string `json:"customer_id" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
```

```
LocationCoordinates *string `json:"location_coordinates,omitempty"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
ValuerID *string `json:"valuer_id,omitempty"`
ApproverID *string `json:"approver_id,omitempty"`
Status StatusType `json:"status" gorm:"default:pending"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Partner *Partners `json:"partner,omitempty" gorm:"foreignKey:PartnerID"`
Agent *Agents `json:"agent,omitempty" gorm:"foreignKey:AgentID"`
Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
Valuer *Valuer `json:"valuer,omitempty" gorm:"foreignKey:ValuerID"`
ValuationInvoices []ValuationInvoices `json:"valuation_invoices"
gorm:"foreignKey:ValuationRequestID"`
}

// ValuationInvoices represents invoices for valuation services
type ValuationInvoices struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
ValuationRequestID string `json:"valuation_request_id" gorm:"not null"`
InvoiceNumber string `json:"invoice_number" gorm:"unique;not null"`
AmountBilled float64 `json:"amount_billed" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:unpaid"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
ValuationRequest ValuationRequests `json:"valuation_request"
gorm:"foreignKey:ValuationRequestID"`
//ValuationPayments []ValuationPayments `json:"valuation_payments"
gorm:"foreignKey:InvoiceNumber;references:InvoiceNumber"`
}

// ValuationPayments represents payments for valuation invoices
type ValuationPayments struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InvoiceNumber string `json:"invoice_number" gorm:"not null"`
ReceiptNumber string `json:"receipt_number" gorm:"unique;not null"`
AmountPaid float64 `json:"amount_paid" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:paid"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
```

```
// Relationships - Note: Reference by InvoiceNumber instead of ID
//ValuationInvoice ValuationInvoices `json:"valuation_invoice"
gorm:"foreignKey:InvoiceNumber;references:InvoiceNumber"`
}

// BeforeCreate hooks for UUID generation
func (c *Customer) BeforeCreate(tx *gorm.DB) error {
if c.ID == "" {
c.ID = uuid.New().String()
}
return nil
}

func (p *Partners) BeforeCreate(tx *gorm.DB) error {
if p.ID == "" {
p.ID = uuid.New().String()
}
return nil
}

func (a *Agents) BeforeCreate(tx *gorm.DB) error {
if a.ID == "" {
a.ID = uuid.New().String()
}
return nil
}

func (vt *ValuationTypes) BeforeCreate(tx *gorm.DB) error {
if vt.ID == "" {
vt.ID = uuid.New().String()
}
return nil
}

func (b *Branches) BeforeCreate(tx *gorm.DB) error {
if b.ID == "" {
b.ID = uuid.New().String()
}
return nil
}

func (vi *VehicleInsurances) BeforeCreate(tx *gorm.DB) error {
if vi.ID == "" {
```

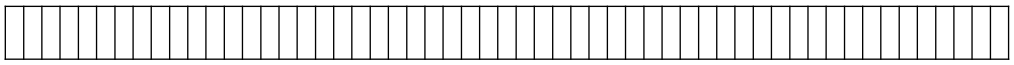
```
vi.ID = uuid.New().String()
}
return nil
}

func (v *Valuer) BeforeCreate(tx *gorm.DB) error {
if v.ID == "" {
v.ID = uuid.New().String()
}
return nil
}

func (vr *ValuationRequests) BeforeCreate(tx *gorm.DB) error {
if vr.ID == "" {
vr.ID = uuid.New().String()
}
return nil
}

func (vi *ValuationInvoices) BeforeCreate(tx *gorm.DB) error {
if vi.ID == "" {
vi.ID = uuid.New().String()
}
return nil
}

func (vp *ValuationPayments) BeforeCreate(tx *gorm.DB) error {
if vp.ID == "" {
vp.ID = uuid.New().String()
}
return nil
}
```



-

Makefile

```
.PHONY: help build test run clean docker-build docker-run migrate seed dev lint fmt deps

# Variables
APP_NAME=user-management-service
DOCKER_IMAGE=aa-core/$(APP_NAME)
DOCKER_TAG=latest
```

```
PORT=8080

# Default target
help:
@echo "Available commands:"
@echo "  build - Build the application"
@echo "  build-all - Build all services"
@echo "  run - Run user management service"
@echo "  run-all - Run all services concurrently"
@echo "  run-auth - Run auth service"
@echo "  run-payment - Run payment service"
@echo "  run-vehicle - Run vehicle service"
@echo "  test - Run tests"
@echo "  docker-dev - Start development environment"
@echo "  clean - Clean build artifacts"

# Build auth service
build-auth:
@echo "Building auth service..."
go build -o bin/auth-service ./cmd/auth-service

# Run auth service
run-auth:
@echo "Starting auth service on port 8083..."
AUTH_SERVICE_PORT=8083 go run ./cmd/auth-service

# Test auth service
test-auth:
@echo "Testing auth service..."
go test ./internal/auth-service/... -v

# Run valuation service
run-valuation:
@echo "Starting valuation service on port 8084..."
VALUATION_SERVICE_PORT=8084 go run ./cmd/valuation-service

# Test auth service
test-valuation:
@echo "Testing valuation service..."
go test ./internal/valuation-service/... -v

# Build vehicle service
build-valuation:
@echo "Building valuation service..."
```

```
go build -o bin/valuation-service ./cmd/valuation-service

# Build user management
build:
@echo "Building $(APP_NAME)..."
go build -o bin/$(APP_NAME) ./cmd/user-management

# Build payment service
build-payment:
@echo "Building payment service..."
go build -o bin/payment-service ./cmd/payment-service

# Build vehicle service
build-vehicle:
@echo "Building vehicle service..."
go build -o bin/vehicle-service ./cmd/vehicle-service

# Build all services
build-all: build build-auth build-payment build-vehicle
@echo "All services built successfully!"

# Run user management service
run:
@echo "Starting $(APP_NAME) on port $(PORT)..."
go run ./cmd/user-management

# Run payment service
run-payment:
@echo "Starting payment service on port 8081..."
PAYMENT_SERVICE_URL=http://localhost:8081 go run ./cmd/payment-service

# Run vehicle service
run-vehicle:
@echo "Starting vehicle service on port 8082..."
VEHICLE_SERVICE_URL=http://localhost:8082 go run ./cmd/vehicle-service

# Run all services with proper startup order
run-all:
@echo "🚀 Starting all services..."
@bash -c ' \
cleanup() { \
echo ""; \
echo "🛑 Shutting down all services..."; \
```

```
[ ! -z "$$USER_PID" ] && kill $$USER_PID 2>/dev/null && echo " ✅ User Management stopped"; \
[ ! -z "$$AUTH_PID" ] && kill $$AUTH_PID 2>/dev/null && echo " ✅ Auth Service stopped"; \
[ ! -z "$$VALUATION_PID" ] && kill $$VALUATION_PID 2>/dev/null && echo " ✅ valuation stopped"; \
[ ! -z "$$PAYMENT_PID" ] && kill $$PAYMENT_PID 2>/dev/null && echo " ✅ Payment Service stopped"; \
[ ! -z "$$VEHICLE_PID" ] && kill $$VEHICLE_PID 2>/dev/null && echo " ✅ Vehicle Service stopped"; \
echo "🏁 All services stopped successfully"; \
exit 0; \
}; \
trap cleanup INT TERM; \
echo " 🔐 Starting Auth Service..."; \
AUTH_SERVICE_PORT=8083 go run ./cmd/auth-service & \
AUTH_PID=$$!; \
sleep 3; \
echo " 📊 Starting User Management Service..."; \
go run ./cmd/user-management & \
USER_PID=$$!; \
sleep 2; \
echo " 💳 Starting Payment Service..."; \
PAYMENT_SERVICE_URL=http://localhost:8081 go run ./cmd/payment-service & \
PAYMENT_PID=$$!; \
echo " 💳 Starting Valuation Service..."; \
VALUATION_SERVICE_URL=http://localhost:8084 go run ./cmd/valuation-service & \
VALUATION_PID=$$!; \
echo " 🚗 Starting Vehicle Service..."; \
VEHICLE_SERVICE_URL=http://localhost:8082 go run ./cmd/vehicle-service & \
VEHICLE_PID=$$!; \
echo ""; \
echo "✅ All services running:"; \
echo " 🔐 Auth Service (PID: $$AUTH_PID) → http://localhost:8083"; \
echo " 📊 User Management (PID: $$USER_PID) → http://localhost:8080"; \
echo " 💳 Payment Service (PID: $$PAYMENT_PID) → http://localhost:8081"; \
echo " 🚗 Vehicle Service (PID: $$VEHICLE_PID) → http://localhost:8082"; \
echo " 🚗 Valuation Service (PID: $$VALUATION_PID) → http://localhost:8084"; \
echo ""; \
echo "🛑 Press Ctrl+C to stop all services"; \
echo ""; \
wait; \
'
```

```
# Service status check
status:
@echo "🔍 Service Status:"
@if lsof -i :8080 >/dev/null 2>&1; then \
echo " 📊 User Management: ✅ Running (port 8080)"; \
else \
echo " 📊 User Management: ❌ Not running"; \
fi
@if lsof -i :8083 >/dev/null 2>&1; then \
echo " 🔐 Auth Service: ✅ Running (port 8083)"; \
else \
echo " 🔐 Auth Service: ❌ Not running"; \
fi
@if lsof -i :8081 >/dev/null 2>&1; then \
echo " 💳 Payment Service: ✅ Running (port 8081)"; \
else \
echo " 💳 Payment Service: ❌ Not running"; \
fi
@if lsof -i :8082 >/dev/null 2>&1; then \
echo " 🚗 Vehicle Service: ✅ Running (port 8082)"; \
else \
echo " 🚗 Vehicle Service: ❌ Not running"; \
fi

# Generate service token for testing
generate-token:
@echo "🔑 Generating service token for payment-service..."
@curl -s -X POST http://localhost:8083/generate-service-token \
-H "Content-Type: application/json" \
-d '{"service_id": "payment-service", "version": "1.0"}' | \
jq -r '.data.service_token' || echo "❌ Failed to generate token (is auth service
running?)"

# Test auth service endpoints
test-auth-endpoints:
@echo "🧪 Testing auth service endpoints..."
@echo "Health check:"
@curl -s http://localhost:8083/health | jq . || echo "❌ Auth service not responding"

# Kill all services
kill-all:
@echo "🚨 Force killing all services..."
@pkill -f "go run ./cmd/user-management" 2>/dev/null || echo " 📊 User
Management: Not running"
```

```
@pkill -f "go run ./cmd/auth-service" 2>/dev/null || echo " 🔐 Auth Service: Not running"
@pkill -f "go run ./cmd/payment-service" 2>/dev/null || echo " 💳 Payment Service: Not running"
@pkill -f "go run ./cmd/vehicle-service" 2>/dev/null || echo " 🚗 Vehicle Service: Not running"
@pkill -f "go run ./cmd/valuation-service" 2>/dev/null || echo " 🚗 Valuation Service: Not running"
@echo "✅ Cleanup complete"

# Rest of your existing targets...
test:
@echo "Running tests..."
go test ./... -v -race -coverprofile=coverage.out
go tool cover -html=coverage.out -o coverage.html

dev:
@echo "Starting development server with hot reload..."
@if command -v air > /dev/null; then \
air; \
else \
echo "Installing air..."; \
go install github.com/cosmtrek/air@latest; \
air; \
fi

clean:
@echo "Cleaning build artifacts..."
rm -rf bin/
rm -f coverage.out coverage.html
docker system prune -f

# Docker development
docker-dev:
@echo "Starting development environment..."
docker compose -f docker-compose.dev.yml up -d

docker-dev-down:
@echo "Stopping development environment..."
docker compose -f docker-compose.dev.yml down

.PHONY: build-auth run-auth test-auth run-all status generate-token test-auth-endpoints kill-all
```